Real-Time Parallel Log / Event Triage Engine

A Regex-to-DFA Compiler and Multi-Core Pipeline for Streaming Log Classification

Adithya Santhosh P
Reg. No. 24BCI0099

Anvesha Tripathi
Reg. No. 24BCT0055

Team 1, Compiler Design Course Project, VIT · Project Report · October 2026

***Abstract*—Servers emit thousands of log lines per second, and the few that matter—repeated failed logins, injection attempts, crashes, leaked personal data—must be found as the lines arrive. This report presents a real-time log triage engine written in C11 around a purpose-built regular-expression compiler. Detection rules are parsed by a recursive-descent parser, converted to a Thompson NFA and compiled by subset construction into combined DFAs that match every rule in one pass over a line. A classifier validates matched numbers with Luhn, Verhoeff and PAN checks, fuses rule evidence into confidences by noisy-OR and masks personal data; a multi-threaded pipeline with lock-free queues and a windowed correlator for brute force, password spraying and error storms complete the system. A synthetic log generator with exact ground truth, a benchmark mode and a runner support the evaluation. The DFA agrees with the C library's regex engine on 10,218 corpus lines and 100,000 generated strings (0 mismatches). On one worker it processes 971,394 lines/s against 74,126 for the library (13.1×), and from 5 to 64 rules the library falls to 14 % of its throughput while the DFA stays at 97 % up to 32 rules. Checksum validation raises PII precision from 0.60 to 1.00 on a set with look-alike numbers. The base paper's process-per-pattern design is 206× slower than the sequential library engine on our rules. The DFA engine does not speed up with more workers because serial stages bound the pipeline; recall on attack-labelled real logs is low because the rule set does not cover those attacks. Both are analysed in Sections 4 and 5.**

***Index Terms*—**regular expressions, finite automata, subset construction, log classification, multicore processing, lock-free queues, intrusion detection.

1. INTRODUCTION

1.1 Motivation

Modern services write logs faster than people can read them. Hidden in the stream are lines that need action: repeated authentication failures, SQL-injection or path-traversal requests, application crashes, slow queries, and personal data that must not be stored in clear. *Triage*—assigning every line a category and a severity, and routing urgent lines to an alert stream—has to keep up with the input rate. Hand-written regular-expression rules are the traditional way to encode this knowledge [1], which makes the matching engine the performance-critical part, and a regex that only checks shape cannot tell a card number from a long identifier.

The base paper given for this course classifies short text inputs by running one regular expression per forked process on separate cores. We treat its design and claims as hypotheses and test them. This project takes a different route: compile all rules into one deterministic automaton, parallelise across input lines rather than across patterns, and add the validation and correlation that raw pattern matching lacks.

1.2 Problem Statement and Objectives

Problem. Given a stream of log lines and at most 64 rules, each a regular expression with a category, a severity and a base score, assign every line its categories, a confidence per category, a primary category and a severity, mask personal data, and detect attacks that only show across several lines—at a rate that keeps up with the input. The matching semantics must equal POSIX extended regular expressions on the supported subset, so that the C library serves as an independent reference, and a rule outside the subset must be reported, never silently approximated.

Objectives. (O1) Build a regex-to-DFA compiler from first principles and verify it against the C library. (O2) Reduce false positives by validating matched numbers and combine evidence into calibrated multi-label output. (O3) Process lines on several cores with a clean hand-off between threads and output identical to the sequential run. (O4) Detect multi-line attacks with sliding windows. (O5) Measure the base paper's design and fairer alternatives under a documented protocol. (O6) Build the benchmark and evaluation tools (synthetic generator with exact ground truth, benchmark mode and runner, evaluation scripts) and use them to measure throughput, scaling, rule-count sensitivity, accuracy and the effect of validation.

1.3 Contributions

1) a **regex front end** for the regular POSIX-ERE subset that rejects everything outside it;
2) a **regex-to-DFA compiler**: Thompson NFA, subset construction with accept masks and automatic rule grouping, and a table-driven matcher;
3) a **classifier** with checksum validators, entity extraction, a noisy-OR confidence model and PII masking;
4) a **parallel pipeline** with pinned worker threads, lock-free single-producer single-consumer queues, a pre-allocated buffer pool and ordered output;
5) a **correlator** for brute force, password spraying, fail-then-success and error storms;
6) a faithful **reproduction and benchmark** of the base paper;
7) an **evaluation framework**: a synthetic generator with exact ground truth, a `--bench` mode, an experiment runner, and tools for labelling samples and computing precision, recall, calibration and inter-annotator agreement.

1.4 Organization

Fig. 1 gives an overview: rules are compiled once at start-up; at run time a reader feeds batches to worker threads, an aggregator and a correlator classify and escalate, and a router writes the outputs; the evaluation harness measures the base-paper designs. Section 2 surveys related work, Section 3 describes each module with pseudocode, Section 4 gives implementation results, Section 5 compares existing and proposed methods, and Section 6 concludes.

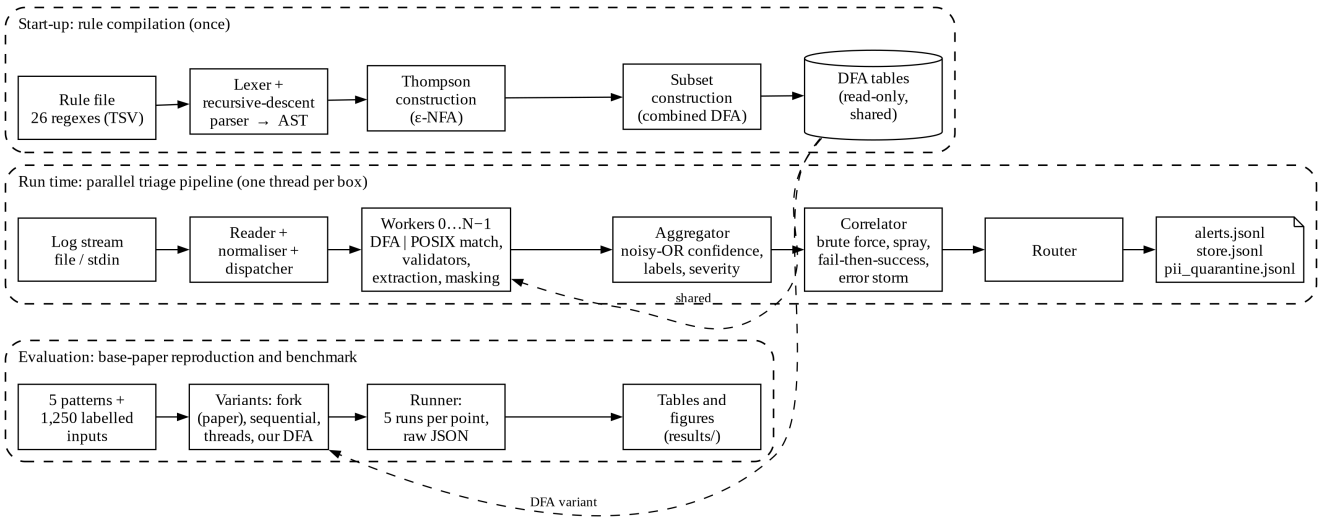


Fig. 1. System architecture: start-up compilation, parallel triage pipeline and evaluation harness.

1.5 Scope and Limitations

This report covers the finished project, source commit `68ea303`. All engineering tasks of the plan are implemented. Two limits matter for reading the results. First, accuracy on the log rules is measured against ground truth that is synthetic (a generator written together with the rules, so it checks consistency, not real-world accuracy) and against attack labels of one public data set (AIT-LDS); on real Loghub lines accuracy is checked only by an AI annotator on 100 lines (clearly not human ground truth), because a hand-labelled sample needs two people labelling independently; the tools for it exist, and no human-labelled number is reported until it is done. Second, the correlator thresholds are the defaults of the fail2ban tool, not tuned values, and all absolute numbers depend on the machine described in Section 4.

2. LITERATURE SURVEY

2.1 Automata Foundations and Regex Compilation

Rabin and Scott introduced nondeterministic finite automata and the decision problems they raise [2], which gives the subset construction its theoretical basis. McNaughton and Yamada gave a procedure that turns a regular expression into an automaton [3]. Thompson described the search algorithm that compiles a regular expression into a nondeterministic matcher [4]; the construction named after him is the one used here. For a fixed set of literal patterns, Aho and Corasick match all patterns in a single pass over the text [5], the same one-pass idea that a combined DFA extends from literals to regular expressions.

2.2 Multi-Pattern Matching and DFA State Explosion

Regular-expression matching for deep packet inspection (DPI) has a large literature. The survey by Xu *et al.* covers applications, algorithms and hardware platforms and identifies the growth in DFA size as the central difficulty [6]. Responses fall into three groups. *Compression:* D^2FA reduces memory with default transitions [7]; differential encoding of DFAs keeps one transition per byte with smaller tables [8]; A-DFA bounds the number of default-transition hops [9]. *Hybrids and decomposition:* hybrid automata combine a DFA with an NFA for complex intrusion-detection patterns [10], and the extensions to counting and back-references are treated in [11]; Liu and Norige decompose expressions to avoid the explosion [12]. *Prefiltering and engineering:* Bloom-filter-assisted matching for intrusion detection [13], and Hyperscan, which splits each expression into literal and automaton parts and uses SIMD matching, reporting an 8.7× speed-up for Snort on a real trace [14]. Backtracking engines have a different weakness: Davis *et al.* found super-linear regular expressions in more than 10,000 JavaScript and Python modules [15], which is the argument for an automaton engine whose cost is linear in the input. Our grouping strategy (split the rule set in halves until each DFA fits a state cap) is deliberately simpler than these techniques.

2.3 Parallel Pattern Matching and Concurrency

Luchaup *et al.* parallelise the matching of *one* input by splitting it into chunks and speculating on the start state of each [16]; Ko *et al.* study a speculative parallel DFA membership test for multicore, SIMD and cloud settings [17], and Mytkowicz *et al.* make finite-state machines data parallel by computing transitions for all possible start states, reporting up to 21× on eight cores [18]. Yang and Prasanna show that large automata behave poorly in the cache of a multicore processor [19]. Log lines are already independent units, so line-level parallelism needs no speculation; this is the option adopted here. Two performance-theory points frame the experiments: Gustafson's scaled-speedup argument [20], and Lamport's work on specifying the interfaces of concurrent program modules [21], which guided the hand-off between pipeline stages. Baumann *et al.* argue that `fork()` has become a poor primitive for modern systems [22], which is relevant to a base-paper design that forks per pattern per input.

2.4 Log Analysis, Datasets and Sensitive Data

He *et al.* survey automated log analysis for reliability engineering [1]. Log parsers extract templates: Drain uses a fixed-depth tree for online parsing [23], and POP parallelises parsing on a cluster [24]. Landauer *et al.* survey clustering for security log analysis and group approaches into overview and filtering, parsing and signature extraction, outlier detection, and sequence anomaly detection [25]; a rule-based triage engine belongs to the filtering and signature group. Public data: Loghub collects system logs from many software systems [26], and the AIT data sets add labelled attack steps [27] produced by a simulation testbed [28]. Event processing platforms are surveyed in [29]. For personal data, the Text Anonymization Benchmark provides an annotated corpus of 1,268 court cases [30] and Lison *et al.* review anonymisation models [31]; check-digit schemes such as the one studied by Gumm [32] allow identifier-like numbers to be validated by arithmetic, which is how the classifier here separates card and Aadhaar numbers from other digit strings.

2.5 Benchmarking Methodology

Hoefler and Belli reviewed 120 papers from three high-performance-computing conferences, found that reporting of parallel performance was often inadequate, and propose rules for it [33]; Roodt *et al.* benchmark regular-expression engines built on Thompson and Glushkov constructions [34]. We follow their spirit: every data point is the median of five runs with minimum and maximum reported, the machine is described, and nothing is tuned to reproduce a published number.

2.6 Summary and Research Gap

TABLE I
SURVEYED WORK AND ITS RELATION TO THIS PROJECT

Work	Technique	Relation to this work
[6]	DPI regex survey	State explosion is the key issue
[7]–[9]	DFA compression	Smaller tables; not needed yet
[10], [12]	Hybrid / decomposed automata	More complex than our grouping
[14]	Literal prefilter + SIMD	Industrial engine, comparison target
[15]	ReDoS study	Motivates linear-time DFA
[16]–[18]	Parallel DFA on one input	We split lines, not inputs
[1], [25]	Log analysis surveys	Rules = filtering / signatures
[26], [27]	Log data sets	Our test data
[30], [32]	Check digits; anonymisation	Validators; masking of PII
[22], [33]	fork cost; benchmarking	Base-paper analysis, method

In the surveyed work, regex automata research targets network payloads, while log research targets parsing, clustering and anomaly detection. A compiled multi-rule DFA applied to per-line triage of heterogeneous logs, combined with checksum validation, evidence fusion, line-parallel execution and windowed correlation, and evaluated against a process-parallel baseline, is the gap this project addresses.

3. PROPOSED SYSTEM

The engine is one C11 program with two phases (Fig. 1). At start-up, the rule file is compiled into DFA tables by the regex front end and the DFA compiler. At run time a pipeline of threads reads log lines, normalises them, matches them with the DFA (or, for comparison, the C library), validates and extracts entities, fuses the evidence into labels, applies windowed correlation, masks personal data and writes JSON-lines records. The tables are read-only after construction and shared by all workers without locks. The evaluation harness reproduces the base paper and measures it together with our DFA. Table II gives the module split.

TABLE II
MODULE SPLIT

Module	Source files	Lines
Regex front end	regex_lex, regex_parse, ast	1,024
NFA/DFA compiler	nfa, dfa, dot, engine_dfa	1,352
Classification, confidence, masking	validators, extract, aggregate, mask	873
Parallel runtime	spsc, pool, worker, dispatch, stats, pipeline, reader, batch, bench	1,976
Correlation and routing	correlate, route, json	845
Common (rules, CLI, engines, normaliser)	main, cli, rules, normalize, timestamp, engine, engine_posix, engine_fork, types	1,502
Reproduction and benchmark	tools/basepaper/ (fork, variants, runner, evaluation)	2,099

The engine totals 7,572 lines of C in 62 files. The front end and the DFA compiler are the compiler core; the classification module, the parallel runtime and the correlator are the new contributions of the run-time system, and all of them carry pseudocode below.

3.1 Regex Front End

The front end turns the text of a rule's regular expression into an abstract syntax tree (AST). The grammar is the POSIX-ERE subset—literals, `.`, bracket expressions, `* + ? {n,m}`, alternation, grouping and the anchors `^ $`—so the same text also runs on the C library, which makes differential testing possible. A lexer produces tokens and the parser has one function per non-terminal (Algorithm 1). Character classes are 256-bit sets, and case-insensitive rules fold the set once at parse time. Every construct outside the regular subset is rejected with the byte offset of the offending character; nothing is approximated, because an approximation would silently change what a rule matches.

Algorithm 1. ParseRegex (recursive descent)

```
function Parse(pattern, icase):
    ast ← ParseAlt(); if input remains then Error(pos)
    return ast
function ParseAlt():
    kids ← [ParseConcat()]
    while next = '|' do append ParseConcat() to kids
    return Alt(kids) if |kids| > 1 else kids[0]
function ParseConcat():
    while next is not '|', ')' or END do append ParseRepeat()
    return Concat(kids) (Empty if none)
function ParseRepeat():
    node ← ParseAtom()
    while next is a quantifier (*, +, ?, {n,m}) do
        if a '?' or '+' follows the quantifier then Error(lazy)
        if {n,m}: need n ≤ m ≤ 1000, else Error
        node ← Repeat(node, n, m)
    return node
function ParseAtom():
    '(' : if next = '?' then Error(lookaround); depth ≤ 100;
        node ← ParseAlt(); expect ')' (else Error(unbalanced))
    '[' : read class into a 256-bit set; '.' : all bytes except \n
    '\' : only escaped metacharacters; \d \b \1 ... → Error
    '^', '$' : only at pattern start / end, else Error
    other : literal byte (set folded if icase)
```

3.2 NFA and Combined-DFA Compiler

The DFA compiler compiles the ASTs of all rules into one automaton. Thompson's construction [4] gives every AST node a small NFA fragment with one entry and one exit (Algorithm 2); a bounded repetition `x{n,m}` is expanded into copies. The fragments of all rules are joined under one start state, and each accept state is tagged with its rule number. Subset construction [2] then builds a DFA whose states are sets of NFA states (Algorithm 3). Three design points matter. First, each DFA state stores an *accept mask*, a 64-bit word whose bit *r* says that rule *r* has matched, so one pass over a line reports every matching rule. Second, to find a match anywhere in the line, the set *U* of start states of all non-anchored rules is added to every transition; anchors are handled by assertion edges that only fire at offset 0 (`^`) or at the end of the line (`$`). Third, bytes that no character set of the NFA distinguishes are merged into byte classes, so the transition function is computed once per class. When a DFA would exceed a state cap, the rule set is halved and each half is built separately, recursively; the group results are OR-ed at match time (Algorithm 4).

Algorithm 2. Thompson construction (fragment = entry, exit)

```
function Build(node) → (s, e):
    Charset: new s, e; s —set→ e
    Concat(k_1…k_n): link exit(k_i) —ε→ entry(k_{i+1})
    Alt: split chain; s —ε→ entry(k_i); exit(k_i) —ε→ e
    Star: s —ε→ entry(k), e; exit(k) —ε→ entry(k), e
    Plus: as Star, but s = entry(k)
    Quest: s —ε→ entry(k), e; exit(k) —ε→ e
    Repeat{n,m}: n copies in sequence, then (m−n) nested optional copies
    ^ , $ : assertion edge, usable at offset 0 / at line end only
for each rule r: (s_r, e_r) ← Build(ast_r); tag e_r accept(r)
start ← new state with ε-edges to every s_r
```

Algorithm 2 leaves an NFA in which many states can be active at once: every ε-edge is followed without reading a byte, and every byte may lead to several states. Subset construction removes this nondeterminism by treating the *set* of NFA states that can be active together as one DFA state. The two operations it needs are the ε-closure of a set (all states reachable through ε-edges alone) and *move* (the states reached from a set by one byte class). A DFA state inherits the rule numbers of every accept state it contains, which is how one state can report several rules at once.

Algorithm 3. Subset construction with accept masks and grouping

```
function BuildDfa(rules, cap):
    U ← ε-closure of the start states of rules not '^'-anchored
    S_0 ← ε-closure of the start states of all rules, '^' allowed
    table ← {S_0 : 0}; work ← [S_0]
    while work not empty do
        S ← pop(work)
        for each byte class c do
            T ← ε-closure(move(S, c)) union U
            if T not in table then
                if |table| = cap then return TOO_BIG
                add T to table; push T on work
            next[S][bytes of c] ← id(T)
        accept[S] ← rules whose accept state is in S
        accept_end[S] ← rules reachable from S by ε / '$' edges
    return tables
function BuildAll(rules):
    if BuildDfa(rules) = TOO_BIG then
        if |rules| = 1 then Error
        BuildAll(first half); BuildAll(second half)
```

The result of Algorithm 3 is a set of flat tables. Each group has a transition table indexed by state and byte class, an accept mask per state, and an accept-at-end mask per state for rules that end in `$`. The tables are written once and never changed, which is why all worker threads can read them without locks. Algorithm 4 shows that matching then reduces to one table lookup and one OR per input byte.

Algorithm 4. DFA match loop

```
function Match(line) → mask:
    mask ← 0
    for each group DFA D do
        st ← 0; m ← accept[0]
        for each byte b of line do
            st ← next[st][b]; m ← m | accept[st]
        mask ← mask | m | accept_end[st]
    return mask
```

3.3 Classification, Confidence and Masking

A regular expression says that a span has the right *shape*; the classifier decides whether the line really belongs to the category. For rules with a validator, each candidate span is checked: the Luhn mod-10 sum for card numbers of 13–19 digits, the Verhoeff dihedral-group check for 12-digit Aadhaar numbers, and the structural rule of the Indian PAN (five letters, four digits, a letter, with a restricted fourth letter). A rule whose validator fails on every span is discarded. A surviving rule r contributes the evidence score s_r = base + 0.25 if a validator passed + 0.15 if a context keyword (for example `card` or `payment`) occurs on the line, clamped to [0, 1]; this follows the tiered pattern, checksum, keyword scheme of data-loss-prevention products. The confidence of a category is the noisy-OR of its rules, $conf(c) = 1 - \Pi(1 - s_i)$: independent weak signals add up, one strong rule is enough. Categories with confidence ≥ τ (default 0.5) become labels, so a line can carry several; the primary label has the highest severity, then the higher confidence. Source address and user name are extracted from matched lines for the correlator, and spans of PII rules are masked before the record is written (`XXXX XXXX 1234` for Aadhaar, `XXXX…1111` for cards). Algorithms 5 and 6 give the validator and the fusion.

Algorithm 5. Luhn validation of a candidate span

```
1  function LuhnOk(span):
2      d ← digits of span without ' ' and '-'; if |d|  [13, 19] return false
3      sum ← 0
4      for i ← 0…|d|−1, counting from the right do
5          x ← d[i]; if i odd then x ← 2x; if x > 9 then x ← x − 9
6          sum ← sum + x
7      return sum mod 10 = 0
```

As a worked example, the digits `4111 1111 1111 1111` pass Algorithm 5 (the weighted sum is a multiple of 10), whereas `4111 1111 1111 1112` fails it, so the second number never becomes a PII label even though both have the shape of a card number. Algorithm 6 then turns the surviving rules into labels. For the passing card number on a line containing the word `payment`, the card rule scores 0.35 + 0.25 (checksum) + 0.15 (keyword) = 0.75, which is above the threshold of 0.5, so the line is labelled PII with confidence 0.75; without the checksum or the keyword the score would stay below the threshold.

Algorithm 6. Evidence fusion (aggregator)

```
1  function Decide(mask, valid, ctx):
2      for each rule r in mask do
3          if r has a validator and r  valid then continue
4          s ← base(r) + 0.25·[validator passed] + 0.15·[ctx contains a
           keyword of r]
5          c ← category(r); prod[c] ← prod[c]·(1 − min(s, 1))
6          sev[c] ← max(sev[c], severity(r))
7      for each category c do conf[c] ← 1 − prod[c]
8      labels ← {c : conf[c] ≥ τ}
9      primary ← argmax over labels by (severity, confidence, fixed
       order SEC>PII>AUTH>ERR>PERF)
10     return (labels, primary, severity(primary))
```

3.4 Parallel Pipeline

The pipeline is a chain of threads connected by lock-free single-producer single-consumer ring buffers (Fig. 1), each with exactly one writer and one reader, so a hand-off needs only acquire and release atomic operations. A reader thread splits the input into lines, fills batches (up to 1,024 lines) and, as dispatcher, hands each batch to a worker by round robin or to the least loaded queue. Each worker is pinned to a core (one thread per physical core before hyper-thread siblings), owns its engine state—for the POSIX engine its own compiled `regex_t` set, for the DFA engine a read-only share of the tables—and classifies the batch in place. All batch buffers come from a pool allocated before the threads start and are recycled through a return ring, so no memory is allocated per line (an allocation-counting test checks this). The aggregator collects worker results, the correlator restores input order before it applies its windows, and the router writes the three output files. Backpressure is blocking by default; `--drop` discards and counts batches instead. Shutdown sends an end-of-input sentinel through every ring and joins all threads; any failure raises a shared stop flag so that no thread blocks forever. Algorithm 7 gives the worker and the ring.

Algorithm 7. SPSC ring hand-off and worker loop

```
1  function Push(ring, x): // producer only
2      while tail − load_acquire(head) = capacity do wait (or fail if
       stopped)
3      slot[tail mod capacity] ← x; store_release(tail, tail + 1)
4  function Pop(ring): // consumer only
5      while head = load_acquire(tail) do wait (or fail if stopped)
6      x ← slot[head mod capacity]; store_release(head, head + 1);
       return x
7  function Worker(in, out):
8      pin to core; build own engine state
9      loop
10         job ← Pop(in); if job = EOF then Push(out, EOF); break
11         for each line in job do
12             mask ← Match(normalised line); valid ← validators on
               spans; extract IP, user; mask PII text
13         Push(out, job)
```

3.5 Attack Correlation

Some attacks are invisible in a single line: one failed login is noise, five from one address in a minute is a brute-force attack. The correlator runs on one thread behind the aggregator, so its state needs no locks. It keeps a hash table keyed by source IPv4 address; each entry holds a ring of the last 64 authentication failures (time and a hash of the user name). The time is the timestamp parsed from the log, so replays of old logs behave as real time. Four detectors use these windows: *brute force* (at least 5 failures from one address within 60 s), *password spraying* (failures against at least 5 distinct users within 300 s), *fail-then-success* (a successful login from an address that is over the brute-force threshold, severity CRITICAL) and *error storm* (at least 100 ERROR lines within 60 s, reported once). The parameter names and defaults of the first two follow fail2ban. Because workers finish out of order, the correlator holds early batches until the next expected sequence number arrives; output is therefore in input order whatever the worker count. An escalation raises the category confidence to 0.95 and reruns the label decision (Algorithm 8).

Algorithm 8. Correlator

```
1  function OnLine(line): // lines arrive in input order
2      if line is an AUTH failure with source address ip then
3          e ← entry(ip); push (time, hash(user)) into e.ring
4          n ← failures in e.ring within 60 s of now
5          if n ≥ 5 and not already reported in this window then
               escalate(BRUTE_FORCE, HIGH, 0.95)
6          u ← distinct users in e.ring within 300 s
7          if u ≥ 5 and not reported then escalate(PASSWORD_SPRAY,
               HIGH)
8      if line is a successful login from ip and ip over the brute-force
           threshold then escalate(FAIL_THEN_SUCCESS, CRITICAL)
9      if line labelled ERROR then count it in a 60 s window;
10         if count ≥ 100 and no storm active then
               escalate(ERROR_STORM, HIGH)
```

3.6 Base-Paper Reproduction

The evaluation harness rebuilds the base paper's program as described and measures it. For each input the program creates a pipe, forks one child per pattern, pins each child to a core, and the child compiles its regular expression, matches it and reports the result through the pipe (Algorithm 9); the parent collects the five records and reaps the children. The paper's input-normalisation pseudocode is reproduced as an option, and its inefficiencies are kept on purpose. For comparison, one benchmark binary offers six variants on the same five patterns: the forking design, a sequential loop that recompiles per input, a sequential loop with precompiled patterns, five threads (one pattern each), data-parallel threads (inputs split across threads), and our combined DFA with split inputs. A runner executes every experiment five times and stores the raw JSON.

Algorithm 9. Base-paper classification (fork per pattern)

```
1  function ClassifyFork(text):
2      create pipe; for i in 0…4 do
3          pid_i ← fork()
4          if child then
5              pin to CPU (i mod ncpu); regcomp(P_i)
6              ok ← regexec(text); regfree
7              write (i, ok, t_compile, t_exec) to pipe; exit
8      for k in 1…5 do
9          read one record; waitpid(pid_k)
10     return bit mask of matched patterns
```

3.7 Benchmark and Evaluation Framework

The evaluation needs inputs of known content and size, measurements that exclude disk and start-up, and scripts that compute accuracy. The *fork engine* (`--engine fork`) runs the base paper's design on our rules: per line it forks one child per rule, each child compiles and matches its rule and reports through a pipe, and the parent collects and reaps the children; its masks equal those of the POSIX engine. The *benchmark mode* (`--bench`) reads the whole input into an anonymous in-memory file before the clock starts, writes the three output sinks to `/dev/null` (records are still formatted), stamps every line when the reader ingests it, records the time until the aggregator has classified it in a log-scale histogram (8 buckets per power of two), and prints one JSON object with throughput, latency percentiles and DFA size. The *generator* mixes four log types (sshd, Apache access and error, application) and injects a configured fraction of lines per category: auth failures, attack payloads, errors, performance warnings, PII with valid checksums, and look-alike numbers that fail the checksum. The number of injected lines is exact, because each line's class is drawn without replacement (the first function of Algorithm 10), and the ground-truth file lists the true labels. Its Luhn and Verhoeff checks are written independently of the engine's, and source addresses are drawn from the whole IPv4 space so that the correlator stays silent and the truth concerns single lines. A *runner* executes named experiments, each point at least five times, interleaved, and logs the machine, the system load before every run and the run-to-run spread; a point whose runs differ by more than 15 % is flagged for repetition. The *evaluation tools* draw a stratified sample of lines for two independent annotators (without showing the engine's prediction), and compute per-category precision, recall and F1, a confusion matrix of primary labels, Cohen's kappa between annotators, and the calibration of the confidence of the primary label by reliability diagram and Expected Calibration Error, ECE = Σ_b $(|B_b|/N)\cdot|\mathrm{acc}(B_b) - \mathrm{conf}(B_b)|$ (Algorithm 10).

Algorithm 10. Exact-count injection and calibration error

```
1  function GenerateLine(left[0…K], total): // left[c] = lines of class c
       still to write
2      x ← uniform integer in [0, total)
3      c ← the class whose cumulative range of left contains x
4      left[c] ← left[c] − 1; total ← total − 1
5      return a line of class c, and its true labels
6  function ECE(lines): // lines with a primary label
7      bin the lines by the confidence of the primary label into 10 bins
8      return Σ_b (|B_b|/N)·|precision(B_b) − mean confidence(B_b)|
```

4. RESULTS AND IMPLEMENTATION

All results come from commit `68ea303`, built with gcc 15.2 and `-O2 -Wall -Wextra -Wpedantic -pthread` without warnings, on a 13th-generation Intel Core i7-13650HX (14 cores) under Linux. The code base has 7,572 lines of engine source, 5,762 lines of tests, 2,099 lines of base-paper tooling and 3,429 lines of other tools. 20 unit-test programs run 364,556 checks with no failure, and the code runs without reports under ThreadSanitizer and AddressSanitizer. Every measurement is stored under `results/`.

4.1 Dataset Description

TABLE III
DATA SETS

Data set	Source	Lines	Use
OpenSSH_2k	Loghub sample	2,000	AUTH rules, correlator
Apache_2k	Loghub sample	2,000	web-server log
Linux_2k	Loghub sample	2,000	syslog / PAM, PII
BGL_2k	Loghub sample	2,000	ERROR rules, error storm
HDFS_2k	Loghub sample	2,000	identifiers vs. PII
synthetic_small	hand-written	218	99 annotated lines: positives, near-misses, correlator cases
Loghub full (4 logs)	Loghub, downloaded	5,485,159	label distribution, correlator
AIT-LDS (3 logs)	AIT v2.0, labelled	8,838	attack ground truth
generated logs	tools/gen, fixed seeds	1,000,000 / 20,000	speed inputs; accuracy truth
labelled set	generated, fixed seed	1,250	five base-paper categories

The first five are the 2,000-line samples published with Loghub [26]. They carry no labels, so for them we report what the engine finds. The hand-written corpus annotates each line with the exact set of rules that must match it, including look-alikes such as a 12-digit order number or the word “errors”, and contains scripted attacks for the correlator. The labelled set of 1,250 inputs is generated from templates with a fixed seed, balanced over e-mail, phone, question, greeting and name plus none, multi-category and near-miss lines; its ground truth describes what a line *is*, not what the regexes match. The full-size Loghub logs (OpenSSH, Apache, Linux, BGL) are downloaded by a script; they have no labels. Of the AIT-LDS v2.0 data set [27] we take one scenario (russellmitchell): the zip archives are 7 to 27 GB, mostly packet captures, so the script reads only the labelled Apache and authentication logs by HTTP range requests and converts the per-line labels. The generated logs are described in Section 3.7; their ground truth is exact by construction.

4.2 Regex Front End

TABLE IV
REJECTED SYNTAX (MESSAGE AND BYTE OFFSET)

Pattern	Compiler message	Offset
`a\d+`	\d shorthand not supported (write [0-9])	1
`a\bb`	\b word boundary not supported	1
`(a)\1`	backreference \1 (not a regular language)	3
`(?=a)b`	"(?" groups: lookaround not supported	0
`a*?b`	lazy / possessive quantifier	2
`a{,3}`	{,n} not supported; write {0,n}	1
`a^b`	'^' only allowed at pattern start	1
`(ab`	unbalanced '('	0
`a{2,1}`	invalid interval, m < n	1

All 26 default rules parse. The parser test has 264 checks over valid and invalid patterns. Table IV shows that every unsupported construct is refused with a message that names a regular alternative where one exists. *Inference:* the front end enforces the regular subset strictly; a rule author is told how to rewrite the rule instead of getting silently different behaviour.

4.3 NFA and DFA Compiler

The 26 rules give a combined NFA of 1,762 states. Table V and Fig. 2 show what the state cap does. Without a cap the combined DFA has 39,764 states and takes 955 ms to build; with the default cap of 20,000 it is split into 2 groups with 7,593 states in total (522 ms), a 5.2× reduction in table states. Tighter caps give more groups: at 500 states the DFA needs 5 groups of 1,680 states in total, built in 23 ms.

TABLE V
DFA SIZE VERSUS STATE CAP (26 RULES)

Cap	Groups	DFA states	Build (ms)
500	5	1,680	23
1,000	5	1,680	48
2,000	4	2,899	90
5,000	3	5,708	209
20,000	2	7,593	522
1,000,000	1	39,764	955

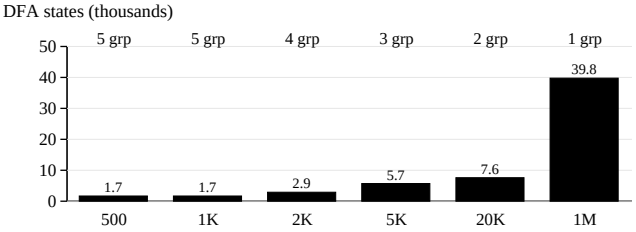


Fig. 2. Total DFA states against the state cap; the label above each bar is the number of groups.

Correctness. The differential tester compares the DFA's rule mask with the C library's for every rule on every line of the 6 corpus files (10,218 lines) and on 100,000 generated strings: **0 mismatches**. The DFA test (169,238 checks) additionally compares the DFA with an NFA simulator, and the NFA test (147,102 checks) compares that simulator with `regexec`. *Inference:* the combined DFA is semantically equal to the reference, and grouping trades a few more table lookups per byte (one pass per group) for much smaller tables.

4.4 Classification, Confidence and Masking

TABLE VI
CLASSIFICATION OUTPUT

File	Lines	Rule hits	Primary label counts
Apache_2k	2,000	0	none
BGL_2k	2,000	402	ERR 395
HDFS_2k	2,000	1	PII 1
Linux_2k	2,000	3	PII 2
OpenSSH_2k	2,000	1,128	AUTH 1086, SEC 39, ERR 2
synthetic_small	218	67	AUTH 18, SEC 16, ERR 10, PII 10, PERF 4

Table VI shows the distribution on real logs: OpenSSH is dominated by authentication failures (1,086 of 2,000 lines) and BGL by errors. The Apache sample produces no hit: its lines are routine requests and none matches a rule, which the rule set is designed to do. The classifier agrees with an independent Python reimplementation of the same specification on primary label, label set, severity, confidence and flags for every line of all six files, and with the annotations of all 99 annotated lines of the hand-written corpus. Validation is the main change from a pattern-only classifier. Table VII compares the number of lines labelled PII before validators existed (commit 8093e0a) and

now.

TABLE VII
LINES LABELLED PII, BEFORE AND AFTER VALIDATION

File	Pattern only	With validation
HDFS_2k	119	1
Linux_2k	29	2
BGL_2k	9	0

Inference. HDFS block identifiers such as `blk_278357163850888` are long digit strings that match the card and Aadhaar shapes; the checksum rejects almost all of them (119 → 1 lines). The few survivors are expected: a random 13–19 digit string passes the Luhn test with probability about 1 in 10, and an all-zero BIOS address in the Linux sample passes trivially. The Linux e-mail hit is a build-system address in a kernel banner; it is a correct match of the PII pattern, which the policy treats as PII. The validator therefore removes false positives but cannot reach zero, which is why confidence is reported (a pattern-only card match scores 0.35, validated 0.60, with a keyword 0.75). Masked values reach the quarantine file and the store; the raw digits are never written.

4.5 Parallel Pipeline

Correctness comes first. A comparison script (`make compare`) runs every corpus file with both engines at 1, 2, 4 and 8 workers, with a small batch size, with the least-loaded dispatcher and with input from a pipe, and requires all three output files to be equal, record by record, to the single-worker run: 72 runs, 0 differences. The single-worker output is in turn equal to the independent Python reference. ThreadSanitizer reports no data race on the test suite, the end-to-end tests or the comparison sweep, and an allocation-counting test shows that the number of allocations does not depend on the input size (it is the same for 100,000 and 500,000 lines at one worker). Table VIII and Fig. 3 give the throughput against the worker count (experiment E2: 1,000,000 generated lines, the default 26 rules, five runs per point, up to 14 workers, the number of physical cores).

TABLE VIII
THROUGHPUT AGAINST WORKER COUNT (MEDIAN OF 5 RUNS)

Workers	DFA (lines/s)	× vs 1	POSIX (lines/s)	× vs 1
1	971,394	1.00	74,126	1.00
2	944,128	0.97	143,578	1.94
4	850,069	0.88	243,226	3.28
6	892,285	0.92	335,315	4.52
8	863,661	0.89	341,089	4.60
10	848,410	0.87	406,776	5.49
12	878,398	0.90	463,006	6.25
14	897,982	0.92	507,421	6.85

Fig. 3 shows the same numbers as speed-up: each curve is divided by the throughput of the same engine with one worker, so a curve on the dotted diagonal would be perfect scaling.

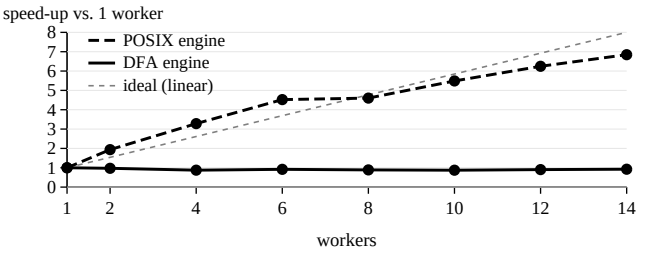


Fig. 3. Speed-up of the whole pipeline against one worker (median of 5 runs).

Inference. The POSIX engine scales: 6.8× at 14 workers (efficiency 0.49), because each line costs 26 library calls and the matching dominates; it keeps an efficiency of at least 0.8 up to 4 workers. The DFA engine does not: its single worker already runs at 971,394 lines/s, and adding workers leaves it flat or slightly lower (0.92× at 14). Matching is so cheap that the serial stages—the reader, the aggregator, the correlator and the router, each on one thread—limit the pipeline at roughly a million lines per second. This is the shape predicted by Amdahl's law, not a defect in the queues: the DFA engine at one worker is faster than the POSIX engine at 14 workers (971,394 against 507,421 lines/s). Moving work out of the serial stages (for example batch-level masking and output formatting) is the next optimisation, and we do not claim DFA scaling beyond what is measured.

4.6 Attack Correlation

TABLE IX
CORRELATOR RESULTS ON THE REAL SAMPLES

File	Brute force	Spray	Error storm	Alerts
OpenSSH_2k	34	5	0	39
BGL_2k	0	0	1	1
synthetic_small	1	1	0	26

Table IX lists the detections. The unit tests (`25,571` checks) cover the cases of the specification: 4 failures give nothing, 5 inside the window give exactly one escalation, 5 spread over more than the window give nothing, and a success after a burst is CRITICAL. On the real OpenSSH sample the correlator reports 34 brute-force and 5 password-spray episodes, all of which appear in the alert file, and one error storm in the BGL sample; a second implementation of the windows in Python agrees on every line. *Inference:* without the correlator these lines are individual MEDIUM authentication failures; escalation turns a pattern over time into one HIGH alert per episode. Whether each episode is a true attack cannot be said without labels; the Loghub OpenSSH sample is known to contain a long scan, and the next step is to score the alerts against the labelled AIT data set.

4.7 Base-Paper Reproduction

TABLE X
LATENCY PER INPUT, MEDIAN (MS)

Input	fork	seq-rec.	seq-pre.	thr-pat.	DFA
`john@example.c…`	193	15.4	0.364	5.23	0.045
`9876543210`	197	15.7	0.321	5.18	0.034
`What is your n…`	201	15.1	0.673	4.97	0.05
`hello everyone`	193	15.7	0.66	5.31	0.042
`John Smith`	205	15.8	0.308	5.12	0.034

The reproduced program classifies one input in 193–205 µs, while a sequential loop that recompiles the five patterns needs

15–16 µs (Table X). Inside each child, `regcomp` takes about 20 µs and `regexec` about 3–5 µs, so process management is most of the 193 µs. The paper's input module also rejects 146 of the 1,250 labelled lines (for example those containing '+' or an apostrophe), because its character whitelist is narrower than the inputs. *Inference:* the paper's central hypothesis—that process overhead dominates—is confirmed, and it works against the paper's own speed-up claim.

4.8 Benchmark Experiments

TABLE XI
ARCHITECTURE COMPARISON (E1)

Config	Engine, workers	Lines	Lines/s	Min–max
C1	fork, 1	2,000	358	343–362
C2	posix, 1	1,000,000	73,844	73,613–74,315
C3	posix, 14	1,000,000	521,485	494,869–526,899
C4	dfa, 14	1,000,000	884,119	873,848–897,005

E1, architecture. Table XI compares the four configurations on the same generated log. The base paper's design, applied to our 26 rules (C1), processes 358 lines/s, which is 206× slower than the sequential POSIX engine (C2, 73,844) and 2,467× slower than the DFA engine with 14 workers (C4, 884,119); C3 (POSIX, 14 workers) reaches 521,485. C1 is measured on 2,000 lines because it is so slow. *Inference:* the ordering C1 << C2 < C3 < C4 expected from the base-paper analysis holds; creating a process per rule per line costs far more than the matching, as the reproduction in Section 4.7 showed for five patterns.

TABLE XII
RULE-COUNT SENSITIVITY (E3)

Rules	POSIX lines/s	DFA lines/s	DFA / POSIX	DFA groups	DFA states	Build (ms)
5	259,850	886,235	3.4×	1	703	4
10	154,796	884,769	5.7×	1	2,554	24
20	81,204	880,042	10.8×	2	4,954	487
32	61,592	863,779	14.0×	2	20,061	816
64	36,829	329,323	8.9×	7	26,517	1,761

Fig. 4 plots the throughput columns of Table XII against the number of rules: the POSIX curve falls steadily, while the DFA curve stays level until the rule set no longer fits in two DFA groups.

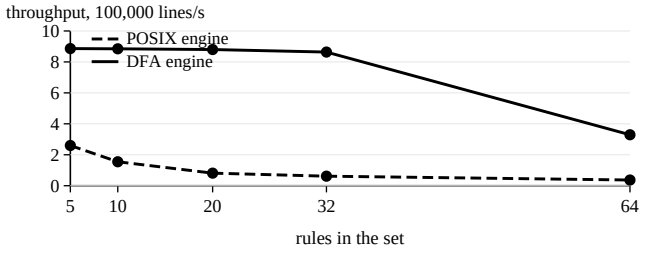


Fig. 4. Throughput against the number of rules, one worker (median of 5 runs).

E3, rule count (the headline experiment). The rule sets of 5, 10, 20, 32 and 64 rules are nested subsets of the default rules plus 38 extra rules (more keywords, attack and PII variants), all inside the supported syntax and all checked against the C library by the differential test. From 5 to 64 rules the POSIX engine falls to 14 % of its throughput (259,850 → 36,829 lines/s), roughly in proportion to the rule count. The DFA engine stays at 97 % of its 5-rule throughput up to 32 rules, then falls to 37 % at 64 rules, where the set needs 7 DFA groups and the line is scanned once per group. *Inference:* the expectation that the DFA stays flat holds only while the rule set fits one or two groups; grouping under a state cap trades memory for a pass per group, and at 64 rules the DFA is still 8.9× faster than the library. The cost on the other side is start-up: building the 64-rule DFAs takes 1.8 s.

TABLE XIII
BATCH SIZE (E4)

Batch	Lines/s	p50 (ms)	p99 (ms)
1	191,318	23.7	61.5
4	563,614	8.4	25.9
16	737,277	6.5	20.0
64	765,928	5.9	16.8
256	787,649	6.5	15.4
1024	806,878	7.1	14.1

E4, batch size. Throughput rises from 191,318 lines/s with one line per batch to 806,878 with 1,024 (Table XIII), because the cost of a queue hand-off is spread over the batch. The latency columns (ingest to aggregated) are for a preloaded input that the reader pushes in as fast as the bounded queues accept it, so they measure queueing, not a paced stream; the p99 latency falls as the batch grows here because the queues drain faster. Latency under a paced load was not measured.

4.9 Accuracy and Validator Study

TABLE XIV
ACCURACY ON GENERATED GROUND TRUTH (E6)

Category	Precision	Recall	F1	Support
Auth Failure	1.000	1.000	1.000	10,000
Security Threat	1.000	1.000	1.000	6,000
Error	1.000	1.000	1.000	10,000
Perf Warning	1.000	1.000	1.000	6,000
Pii	1.000	1.000	1.000	6,000

E6, accuracy. On 200,000 generated lines the engine's labels equal the ground truth on every line (Table XIV, exact label-set match 1.000). This is a consistency check—the generator and the rules were written together—and it shows that the validators reject all look-alike numbers and that no normal line is labelled; it is not an estimate of accuracy on real logs. The confidence of the primary label is under-confident: ECE = 0.37, because the base scores (0.35 weak, 0.6 strong) were set conservatively and every labelled line is correct (observed precision 1.00 in each bin).

TABLE XV
ATTACK DETECTION ON AIT-LDS (E6)

Log	Lines	Attack	Detected	False alarms
auth.log	272	8	1	0
access.log.2	8,530	7,695	6	0
error.log.2	36	36	0	0

On the AIT-LDS scenario, where a line counts as detected if it is labelled security threat or authentication failure, the engine detects 7 of 7,739 labelled attack lines (0.09 %) with 0 false alarms (Table XV). *Inference:* the rule set has no rule for most of the attacker's requests, which are WordPress scans and an upload exploit with ordinary-looking URLs; what it flags is correct, but recall is very low. The rules were not changed after seeing these numbers, so that they are not tuned on the evaluation data. On the full Loghub logs, for which there is no ground truth, the correlator reports 13,412 brute-force and 284 password-spray episodes in the 655,147-line OpenSSH log and

one error storm in BGL, and labels 985,785 BGL lines as errors.

TABLE XVI
AI-ANNOTATOR CHECK ON REAL LOG LINES (E6)

Category	Precision	Recall	F1
Auth Failure	1.000	0.688	0.815
Security Threat	0.400	0.025	0.047
Error	1.000	0.002	0.003
Perf Warning	—	—	—
Pii	0.800	1.000	0.889

E6, AI-annotator check on real lines. To measure accuracy on real logs while the human labelling is pending, a stratified sample of 100 lines from the full OpenSSH, Apache and Linux logs (20 from each stratum of the engine's primary label, plus unlabelled lines) was labelled by an AI annotator (Claude), from the lines and written criteria only, without seeing the engine's answers. This is **not human ground truth**: the annotator also wrote the rules, and several labels are judgement calls (for example, a line that states repeated failed logins counts as a threat). Strata are weighted back to the full logs, so the figures are rough. Table XVI shows high precision for failed logins (1.000) but a recall of 0.688: disconnect lines that report an authentication failure are missed. The recall for errors is 0.002, because the error rule looks for the uppercase level word and misses `error:` lines of sshd. PII precision is 0.800: kernel boot lines with an all-zero memory address pass the card checksum. *Inference:* on real logs the rule set is precise where it fires and misses categories it was not written for; labels from two human annotators, and from a second model, are the planned replacement for this check (`docs/llm_labelling_instructions.md`).

TABLE XVII
VALIDATOR EFFECT ON PII LABELS (E7)

Validators	PII precision	PII recall	False PII
off	0.600	1.000	4,000
on	1.000	1.000	0

Table XVII shows the controlled comparison: the generated set contains 4,000 numbers that look like cards or Aadhaar numbers but fail their checksum, and without validators every one of them becomes a false PII label. The same switch applied to the real logs, for which no labels exist, is shown in Table XVIII and counts the lines labelled PII.

TABLE XVIII
LINES LABELLED PII ON THE FULL REAL LOGS

Log	Lines	Off	On
OpenSSH	655,147	163	0
Linux	25,567	105	40
BGL	4,747,963	12,279	1,175
HDFS 2k	2,000	119	1

E7, validators. With the checksum validators switched off (a pattern match is a label), PII precision on the generated set is 0.60, with 4,000 false labels on look-alike numbers; with them it is 1.00 at the same recall (Table XVII). On the real logs the validators remove most PII labels (Table XVIII): BGL 12,279 → 1,175 lines, OpenSSH 163 → 0, HDFS 119 → 1. Without labels we cannot say how many of the remaining lines are true; a random 13–19 digit string passes the Luhn test about one time in ten, so some false labels survive. *Inference:* the validators are what makes the PII category usable on logs full of identifiers.

5. PERFORMANCE AND METRICS

5.1 Matching Engine: Existing versus Proposed

We compare the C library's POSIX engine (one `regexec` per rule, the approach of the base paper's matching step) with the proposed combined DFA. Both run the same 26 rules inside the same pipeline with one worker on 1,000,000 generated log lines, output written to `/dev/null`, with rule compilation excluded from the timing (`--bench`); five runs per engine, interleaved with the other points of the scaling experiment.

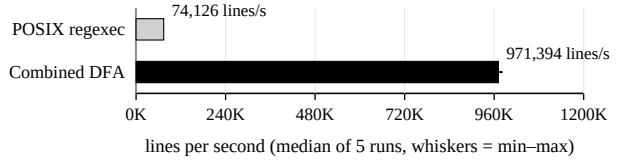


Fig. 5. End-to-end throughput of the two engines on real log rules, one worker.

The DFA engine sustains 971,394 lines/s and the POSIX engine 74,126 lines/s, a factor of **13.1**; the spread between the five runs is at most 7 % of the median. *Inference:* in a loop of 26 library calls every line is scanned once per rule, and the library's cost grows with the rule count (Section 4.8); the DFA scans the line once per group. The factor is a lower bound on the matching speed-up, because the measurement includes the unchanged reader, classifier and output. The price is the start-up cost: building the DFAs takes 535 ms against 15 ms for compiling the library's regexes, which is repaid after about 42 thousand lines.

Across the rule counts of experiment E3 the DFA engine is 3.4× faster than POSIX with 5 rules, 14.0× with 32 and 8.9× with 64; with 14 workers it is 1.70× the POSIX engine (Table XI).

5.2 Base-Paper Design versus Alternatives

Fig. 6 compares six designs on the five-pattern task for a batch of 10,000 inputs (median of five runs). The run-to-run spread (maximum minus minimum, relative to the median) is at most 5 % for 4 designs and 45 % and 92 % for the two fastest, whose 10,000-input runs last only a few milliseconds or less and are dominated by timing noise; the scaling runs therefore use longer runs, up to 10 million inputs.

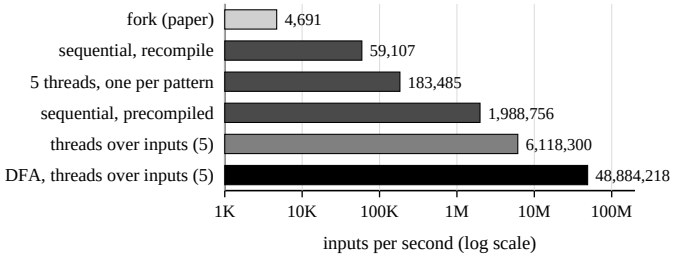


Fig. 6. Throughput of six designs for 10,000 inputs (existing: first row; proposed: last row).

The forking design reaches 4,691 inputs/s, which is 12.6× slower than a sequential loop that recompiles and 424× slower than one with precompiled patterns. Splitting the *inputs* across five threads gives 3.1× the sequential precompiled loop, and our DFA with the same split 24.6×. The fork design peaks at 1.72× and per-pattern threads at 2.54× when cores are added, because five patterns cannot give more than five-fold parallelism and the setup cost is paid per input; data-parallel threads reach 4.83× and the DFA variant 5.29× at six cores. *Inference:* the base paper's design cannot deliver near-linear scaling on this hardware; parallelism over input items does, which is the model

our pipeline uses.

5.3 Accuracy: Existing versus Proposed

On the labelled set of 1,250 inputs we compare the base paper's pipeline (its input module followed by `regexec`; *existing*) with matching on the unmodified text with the same five patterns, as done by the DFA variant (*proposed*). The DFA gives the same masks as `regexec` on every line (asserted by a test), so any difference comes from the paper's input module. Table XIX and Fig. 7 give precision (P), recall (R) and F1 per category.

TABLE XIX
ACCURACY ON THE LABELLED SET

Category	P exist.	R exist.	F1 exist.	P prop.	R prop.	F1 prop.
Email	0.99	0.75	0.85	0.99	1.00	0.99
Phone	0.88	0.67	0.76	0.91	0.87	0.89
Question	1.00	0.76	0.87	1.00	0.77	0.87
Greeting	1.00	0.78	0.88	1.00	0.70	0.83
Name	—	0.00	0.00	0.77	0.67	0.71
Exact match			0.698			0.798
Label accuracy			0.933			0.960

Fig. 7 shows the F1 columns of Table XIX side by side for the two pipelines, which makes the effect of the paper's input module on each category visible at a glance.

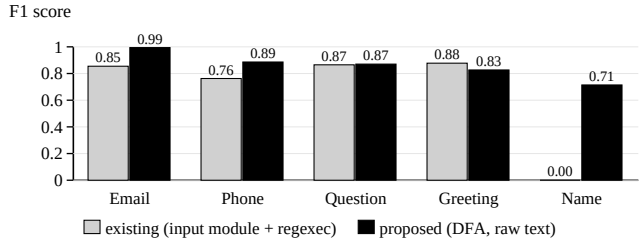


Fig. 7. F1 per category, existing pipeline versus proposed matching.

With the proposed matching, exact-set accuracy is 0.798 against 0.698 for the existing pipeline, and label accuracy 0.960 against 0.933. The existing input module lowercases text, so the name pattern, which needs capital letters, can never match (recall 0.00), and it rejects 146 inputs. Without it, e-mail is nearly perfect (F1 0.99) while question, greeting and name are lower (0.87, 0.83, 0.71): structured formats are easier for regular expressions than natural language, as the paper claims (absolute values depend on our templates). For the log rules themselves, precision against human labels is not yet measured; the evidence so far is the validator effect of Table VII (119 to 1 PII labels on HDFS), agreement with the reference classifier on all 10,218 lines, and exact agreement with the 99 annotated lines. *Inference:* the proposed approach does not lose accuracy relative to the library; any accuracy gain comes from not applying a lossy input stage and from validating matched numbers.

5.4 Summary of Metrics

TABLE XX
EXISTING VERSUS PROPOSED

Metric	Existing	Proposed
Throughput, 26 rules, 1 worker	74,126 lines/s (POSIX)	971,394 lines/s (DFA)
Throughput, 14 workers	521,485 lines/s (POSIX)	884,119 lines/s (DFA)
Throughput, 64 rules, 1 worker	36,829 lines/s	329,323 lines/s
Paper's design on our rules	358 lines/s (fork)	73,844 lines/s (sequential)
Inputs/s, 5 patterns, batch 10K	4,691 (fork)	48,884,218 (DFA)
Mask agreement with regexec	(reference)	0 mismatches
PII precision, look-alike set	0.60 (no validators)	1.00 (validators)
PII labels on HDFS_2k	119 (pattern only)	1 (validated)
Exact-set accuracy, 5 categories	0.698	0.798

5.5 Comparison with the Base Paper

Table XXI compares the base paper provided for the course with this work: the design, what the paper reports, and what we measured. The paper's figures are quoted from the paper; ours come from the saved runs in `results/` (Sections 4 and 5).

TABLE XXI
BASE PAPER VERSUS THIS WORK

Aspect	Base paper	This work
Task	Classify short texts into 5 classes (e-mail, phone, question, greeting, name)	Classify log lines into 5 categories (authentication failure, security threat, error, performance, PII), several labels per line
Input	Typed or batched short texts; its own 1,250 samples (data not available)	Streams: files and pipes; real logs up to 4,747,963 lines; generated logs
Matching	One C-library regex per pattern; 5 fixed patterns	Own regex → NFA → DFA compiler; all rules in one pass; 26 rules (up to 64); library and fork engines kept as baselines
Rule syntax	Any library regex, unchecked	Regular subset only; other syntax rejected with its position
Parallelism	One process forked per pattern per input, pinned to cores	Threads over lines: pinned workers, lock-free queues, pooled buffers, output in input order
Input handling	Own input module (whitelist, lowercasing); rejects 146 of our 1,250 test lines	URL-decoding and control-character stripping; nothing rejected
Decision	Single winner by weights 0.4 / 0.3 / 0.3; confidence from completeness, complexity and history	Noisy-OR of rule evidence (base score, checksum, keyword); a label if confidence ≥ 0.5; several labels
Match checks	None	Luhn, Verhoeff and PAN checksums; masking of personal data
Across lines	None	Correlator: brute force, password spraying, fail-then-success, error storm
Reported speed	3.7 ms parallel vs 8.9 ms sequential per input; 2.4× speed-up	Paper's design rebuilt: 193–205 µs per input, 206× slower than a sequential library loop on our rules; our engine 971,394 lines/s (about 1.0 µs per line), 13.1× the library engine

Aspect	Base paper	This work
Scaling	Near-linear to 6 cores; 5.62× at 8	Paper's design at most 1.72×; threads over inputs 4.83× at 6 cores; our POSIX pipeline 6.85× at 14 workers; DFA pipeline flat (serial stages)
Accuracy	96.3 % overall on its own samples	Paper's task on our set: exact match 0.798 without its input module, 0.698 with it. Log rules: 1.000 on generated truth (consistency only); AIT recall 0.09 %; AI-annotator check on real lines (Section 4.9)
Evidence	Results tables; code and data not provided	Code, seeds, raw JSON of every run, tests and sanitizer runs in the repository
Paper's claims	Speed-up, scaling, latency, accuracy, conflict resolution	Speed-up, scaling, latency: not reproduced; batch effect: partly; structured formats more accurate: supported by trend; conflict resolution: not tested

6. CONCLUSION

We built a real-time log triage engine around a regex-to-DFA compiler written from first principles, and the tools to measure it. The compiler agrees with the C library on 10,218 corpus lines and 100,000 random strings. On 26 rules the DFA engine is 13.1× faster than the library on one worker; the library's throughput falls in proportion to the rule count while the DFA's stays within 3 % up to 32 rules and falls at 64, where 7 DFA groups are needed. Checksum validation raises PII precision from 0.60 to 1.00 on a set with look-alike numbers and removes most PII labels from the real logs, the multi-threaded pipeline reproduces the sequential output exactly on 72 comparison runs, and the correlator turns thousands of failed-login bursts into single alerts. The base paper's process-per-pattern design is 206× slower than the sequential library engine on our rules and 13× slower than a sequential loop on its own five patterns, while parallelism over inputs scales.

The measurements also show limits. DFA start-up takes 535 ms for 26 rules and 1.8 s for 64; the DFA engine's pipeline throughput is capped near a million lines per second by the serial stages and does not rise with workers; confidences are conservative (ECE 0.37); and on attack-labelled real logs the recall is 0.09 % because the rule set has no rules for those attacks. Accuracy against human labels on real log lines is not yet measured: an AI annotator's labels on 100 real lines show precision 1.000 but recall 0.688 for failed logins and near zero for sshd error lines, and the sampling and evaluation tools are ready for two human annotators. Further work is a faster serial stage, a rule set informed by public attack signatures, calibration of the base scores on labelled data, and measurements under a paced load.

REFERENCES

[1] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, "A survey on automated log analysis for reliability engineering," *ACM Comput. Surv.*, vol. 54, no. 6, Art. no. 130, 2021, doi: 10.1145/3460345.

[2] M. O. Rabin and D. Scott, "Finite automata and their decision problems," *IBM J. Res. Develop.*, vol. 3, no. 2, pp. 114–125, Apr. 1959, doi: 10.1147/rd.32.0114.

[3] R. McNaughton and H. Yamada, "Regular expressions and state graphs for automata," *IRE Trans. Electron. Comput.*, vol. EC-9, no. 1, pp. 39–47, Mar. 1960, doi: 10.1109/TEC.1960.5221603.

[4] K. Thompson, "Programming techniques: Regular expression search algorithm," *Commun. ACM*, vol. 11, no. 6, pp. 419–422, Jun. 1968, doi: 10.1145/363347.363387.

[5] A. V. Aho and M. J. Corasick, "Efficient string matching: An aid to bibliographic search," *Commun. ACM*, vol. 18, no. 6, pp. 333–340, Jun. 1975, doi: 10.1145/360825.360855.

[6] C. Xu, S. Chen, J. Su, S. M. Yiu, and L. C. K. Hui, "A survey on regular expression matching for deep packet inspection: Applications, algorithms, and hardware platforms," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 4, pp. 2991–3029, 2016, doi: 10.1109/COMST.2016.2565660.

[7] S. Kumar, S. Dharmapurikar, F. Yu, P. Crowley, and J. Turner, "Algorithms to accelerate multiple regular expressions matching for deep packet inspection," in *Proc. ACM SIGCOMM*, 2006, pp. 339–350, doi: 10.1145/1159913.1159952.

[8] D. Ficara, A. Di Pietro, S. Giordano, G. Procissi, F. Vitucci, and G. Antichi, "Differential encoding of DFAs for fast regular expression matching," *IEEE/ACM Trans. Netw.*, vol. 19, no. 3, pp. 683–694, Jun. 2011, doi: 10.1109/TNET.2010.2089639.

[9] M. Becchi and P. Crowley, "A-DFA: A time- and space-efficient DFA compression algorithm for fast regular expression evaluation," *ACM Trans. Archit. Code Optim.*, vol. 10, no. 1, Art. no. 4, 2013.

[10] M. Becchi and P. Crowley, "A hybrid finite automaton for practical deep packet inspection," in *Proc. ACM CoNEXT*, 2007, pp. 1–12.

[11] M. Becchi and P. Crowley, "Extending finite automata to efficiently match Perl-compatible regular expressions," in *Proc. ACM CoNEXT*, 2008, doi: 10.1145/1544012.1544037.

[12] A. X. Liu and E. Norige, "A de-compositional approach to regular expression matching for network security," *IEEE/ACM Trans. Netw.*, vol. 27, no. 6, pp. 2179–2191, Dec. 2019, doi: 10.1109/TNET.2019.2941920.

[13] S. Dharmapurikar and J. W. Lockwood, "Fast and scalable pattern matching for network intrusion detection systems," *IEEE J. Sel. Areas Commun.*, vol. 24, no. 10, pp. 1781–1792, Oct. 2006.

[14] X. Wang *et al.*, "Hyperscan: A fast multi-pattern regex matcher for modern CPUs," in *Proc. 16th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2019, pp. 631–648.

[15] J. C. Davis, C. A. Coghlan, F. Servant, and D. Lee, "The impact of regular expression denial of service (ReDoS) in practice: An empirical study at the ecosystem scale," in *Proc. ESEC/FSE*, 2018, pp. 246–256, doi: 10.1145/3236024.3236027.

[16] D. Luchaup, R. Smith, C. Estan, and S. Jha, "Speculative parallel pattern matching," *IEEE Trans. Inf. Forensics Security*, vol. 6, no. 2, pp. 438–451, Jun. 2011, doi: 10.1109/TIFS.2011.2112647.

[17] Y. Ko, M. Jung, Y.-S. Han, and B. Burgstaller, "A speculative parallel DFA membership test for multicore, SIMD and cloud computing environments," *Int. J. Parallel Program.*, vol. 42, pp. 456–489, 2014, doi: 10.1007/s10766-013-0258-5.

[18] T. Mytkowicz, M. Musuvathi, and W. Schulte, "Data-parallel finite-state machines," in *Proc. ASPLOS*, 2014, pp. 529–542, doi: 10.1145/2541940.2541988.

[19] Y.-H. E. Yang and V. K. Prasanna, "Robust and scalable string pattern matching for deep packet inspection on multicore processors," *IEEE Trans. Parallel Distrib. Syst.*, vol. 24, pp. 2283–2292, 2013, doi: 10.1109/TPDS.2012.217.

[20] J. L. Gustafson, "Reevaluating Amdahl's law," *Commun. ACM*, vol. 31, no. 5, pp. 532–533, May 1988, doi: 10.1145/42411.42415.

[21] L. Lamport, "Specifying concurrent program modules," *ACM Trans. Program. Lang. Syst.*, vol. 5, no. 2, pp. 190–222, Apr. 1983.

[22] A. Baumann, J. Appavoo, O. Krieger, and T. Roscoe, "A fork() in the road," in *Proc. Workshop Hot Topics Oper. Syst. (HotOS)*, 2019, doi: 10.1145/3317550.3321435.

[23] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2017, doi: 10.1109/ICWS.2017.13.

[24] P. He, J. Zhu, S. He, J. Li, and M. R. Lyu, "Towards automated log parsing for large-scale log data analysis," *IEEE Trans. Dependable Secure Comput.*, 2018.

[25] M. Landauer *et al.*, "System log clustering approaches for cyber security applications: A survey," *Comput. Secur.*, vol. 92, Art. no. 101739, 2020, doi: 10.1016/j.cose.2020.101739.

[26] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," in *Proc. IEEE Int. Symp. Softw. Rel. Eng. (ISSRE)*, 2023.

[27] M. Landauer, F. Skopik, M. Frank, W. Hotwagner, M. Wurzenberger, and A. Rauber, "Maintainable log datasets for evaluation of intrusion detection systems," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 4, pp. 3466–3482, 2023.

[28] M. Landauer, F. Skopik, M. Wurzenberger, W. Hotwagner, and A. Rauber, "Have it your way: Generating customized log datasets with a model-driven simulation testbed," *IEEE Trans. Rel.*, vol. 70, no. 1, pp. 402–415, Mar. 2021.

[29] M. Dayarathna and S. Perera, "Recent advancements in event processing," *ACM Comput. Surv.*, vol. 51, no. 2, pp. 33:1–33:36, 2018, doi: 10.1145/3170432.

[30] I. Pilán, P. Lison, L. Øvrelid, A. Papadopoulou, D. Sánchez, and M. Batet, "The Text Anonymization Benchmark (TAB): A dedicated corpus and evaluation framework for text anonymization," *Comput. Linguist.*, vol. 48, no. 4, pp. 1053–1101, Dec. 2022, doi: 10.1162/coli_a_00458.

[31] P. Lison, I. Pilán, D. Sánchez, M. Batet, and L. Øvrelid, "Anonymisation models for text data: State of the art, challenges and future directions," in *Proc. 59th Annu. Meeting Assoc. Comput. Linguistics (ACL-IJCNLP)*, 2021, pp. 4188–4203, doi: 10.18653/v1/2021.acl-long.323.

[32] H. P. Gumm, "A new class of check-digit methods for arbitrary number systems," *IEEE Trans. Inf. Theory*, vol. 31, no. 1, pp. 102–105, Jan. 1985, doi: 10.1109/TIT.1985.1056991.

[33] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, 2015, Art. no. 73, doi: 10.1145/2807591.2807644.

[34] A. Roodt, B. K. M. Watling, W. Bester, B. van der Merwe, S. Sung, and Y.-S. Han, "Benchmarking regular expression matching," in *Implementation and Application of Automata (CIAA 2024)* (Lecture Notes in Computer Science, vol. 15015). Cham, Switzerland: Springer, 2024.